

Git From Zero to Confident

A Complete Beginner-to-Intermediate Self-Study Course

Personal teacher + textbook + workbook + practice system + roadmap, all in one document. Built for a self-taught programmer who already thinks in code — this course explains Git from first principles, connects every idea to the next, and pushes you toward solving problems independently rather than memorizing commands.

What you'll be able to do when you finish
Explain what Git is and why version control exists, in your own words
Use the core daily workflow: init, add, commit, status, log, diff, branch, merge
Work confidently with remotes: clone, fetch, pull, push, and GitHub-style collaboration
Resolve merge conflicts calmly instead of panicking
Choose between merge and rebase, and undo mistakes safely (reset, revert, reflog)
Read a commit history and reconstruct what happened on a project
Apply a sane branching workflow to a real, multi-day project

How To Use This Course

This is not a book to read start to finish. It's a system. Reading alone barely builds any real understanding — your brain retains what it has to *retrieve*, *use*, and *explain*, far more than what it merely re-reads.

The Learning Loop

Every lesson is designed to be worked through using this loop:

- 1 **Learn** — Read the explanation once, slowly. Don't rush to the exercises.
- 2 **Recall** — Close the lesson. Try to say the core idea out loud or write it from memory.
- 3 **Practice** — Actually type the commands in a real terminal. Typing beats copy-pasting.
- 4 **Explain** — Answer the "Can You Explain It?" prompts in your own words, no peeking.
- 5 **Review** — When you hit a Review Session, take it seriously — it mixes old and new material on purpose.
- 6 **Apply** — Build the projects. This is where scattered facts turn into a real skill.

IMPORTANT

Do not skip the STOP READING tasks. They exist because reading about a command and running that command build completely different kinds of memory. A course you only read will feel understood and then evaporate a week later.

What You Need Before Starting

- A computer with a terminal (macOS Terminal, Linux shell, or Git Bash / PowerShell on Windows).
- Git installed — Lesson 2 walks through this if you haven't yet.
- No prior Git or version-control knowledge required. Basic comfort typing commands in a terminal is enough.

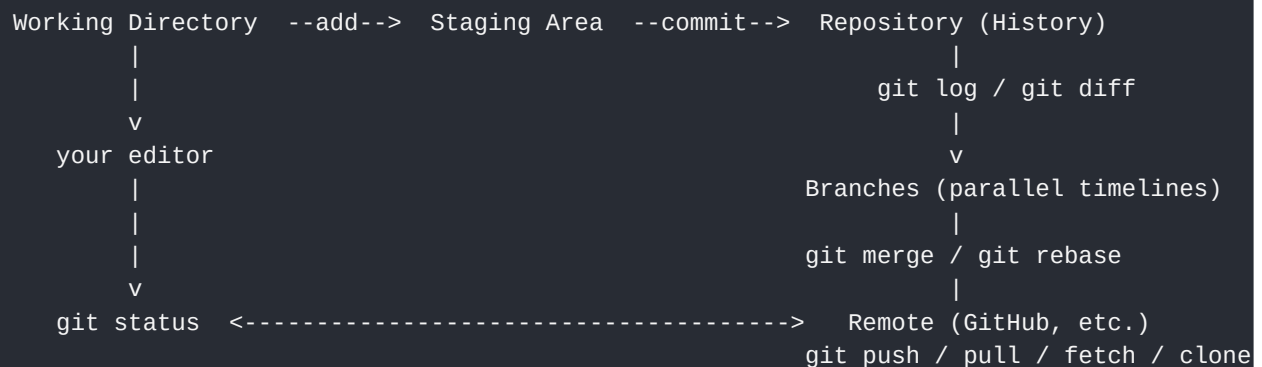
Course Roadmap

Git has a small number of core ideas that combine to produce everything it can do. The roadmap below is ordered so nothing is used before it's explained.

Level	Focus	You'll Come Away Able To...
0	Prerequisites	Understand what a terminal, file, and folder are, and why version control exists at all
1	Foundations	Turn a folder into a Git repository and record your first changes
2	Core Concepts	Branch, merge, and work with a remote repository like GitHub
3	Practical Usage	Use Git the way real projects actually use it day to day
4	Problem Solving	Diagnose and fix common Git situations without panicking
5	Intermediate Concepts	Rebase, undo mistakes safely, and manage releases with tags
6	Mini Projects	Combine everything into small, realistic exercises
7	Final Challenge	Run a multi-day, multi-branch project entirely on your own

Concept Map — How the Pieces Connect

Concept Map



Every lesson from here on will point back to boxes in this diagram so you always know where the new idea fits.

Learning Progress Tracker

Rate your confidence honestly after each lesson: 1 = I don't understand it yet, 5 = I could teach it to someone else. Revisit this page as you go.

#	Topic	Status	Confidence (1-5)
0	Prerequisites: terminal, files, why version control	☐	___
1	What Git Is & Why Version Control Exists	☐	___
2	Installing & Configuring Git	☐	___
3	The Core Workflow: init, add, commit, status, log	☐	___
4	Branching	☐	___
5	Merging & Merge Conflicts	☐	___
6	Remotes: clone, push, pull, fetch	☐	___
7	Everyday Workflow: diff, .gitignore, stash	☐	___
8	Rebasing vs Merging	☐	___
9	Undoing Things: reset, revert, checkout, reflog	☐	___
10	Tags & Releases	☐	___

Table of Contents

Course Introduction

How To Use This Course · Roadmap · Progress Tracker

Level 0 — Prerequisites

Lesson 0: Files, Folders, and the Terminal

Level 1 — Foundations

Lesson 1: What Git Is & Why Version Control Exists

Lesson 2: Installing & Configuring Git

Lesson 3: The Core Workflow — init, add, commit, status, log

Review Session 1 · Knowledge Checkpoint 1

Level 2 — Core Concepts

Lesson 4: Branching

Lesson 5: Merging & Merge Conflicts

Lesson 6: Remotes — clone, push, pull, fetch

Mini Project 1: Personal Notes Repo

Review Session 2 · Knowledge Checkpoint 2

Level 3 — Practical Usage

Lesson 7: The Everyday Workflow — diff, .gitignore, stash, feature branches

Level 4 — Problem Solving

Problem-Solving Lab: Diagnosing Real Git Situations

Level 5 — Intermediate Concepts

Lesson 8: Rebasing vs Merging

Lesson 9: Undoing Things — reset, revert, checkout, reflog

Lesson 10: Tags & Releases

Intermediate Project: Simulated Team Collaboration

Review Session 3 · Knowledge Checkpoint 3

Level 6 — Mini Projects

Mini Project 2: Fix the Broken History

Level 7 — Final Challenge

Final Project: Multi-Branch Feature Release

Course Completion

Final Exam (6 parts) · Answer Key

"Can I Actually Do This?" Checklist

Glossary

Cheat Sheet

What To Learn Next

LEVEL 0 | Prerequisites

What you should be comfortable with before Lesson 1

Lesson 0: Files, Folders, and the Terminal

1. What You Will Learn

- What a terminal (command line) is and why programmers use it
- What a directory (folder) and a path are
- The handful of navigation commands Git assumes you already know

2. Why This Matters

Git is a command-line tool at its core. Before learning Git commands, you need to be able to move around your file system by typing instead of clicking. If this part is shaky, every later lesson will feel harder than it needs to be — not because Git is hard, but because navigating to the right folder is hard.

3. The Simple Explanation

A **terminal** is a text window where you type commands instead of clicking icons. Think of it like texting instructions directly to your computer instead of pointing at pictures of what you want. A **directory** is just the command-line word for "folder." A **path** is the address of a file or folder — like a street address, but for your hard drive.

4. Key Terms

Terminal / Command line

Simple definition: A text-based program for typing commands to your computer.

In plain English: Instead of double-clicking folders, you type where you want to go.

Example: On macOS it's called Terminal; on Windows, Git Bash or PowerShell; on Linux, usually just "terminal."

Working directory

Simple definition: The folder your terminal is currently "standing in."

In plain English: Every terminal command runs relative to whatever folder you're currently in.

Example: If you're in `/Users/gelán/projects`, that's your working directory.

Path

Simple definition: The location of a file or folder, written as text.

In plain English: Absolute paths start from the very top (`/home/ . . .`); relative paths start from where you already are.

Example: `./notes.txt` means "notes.txt in the current folder."

5. How It Works — The Commands You Need

Terminal basics (macOS/Linux/Git Bash syntax)

```
pwd          # Print Working Directory – "where am I right now?"
ls           # List – show files/folders here (Windows: dir)
cd my-folder # Change Directory – move into my-folder
cd ..        # Move up one level (to the parent folder)
mkdir my-project # Make a new directory called my-project
```

TIP

On Windows, PowerShell understands `ls`, `pwd`, and `mkdir` too — but Git Bash is recommended for this course because its commands match the examples exactly.

6. Practical Example

Try this sequence right now:

```
pwd
mkdir git-course
cd git-course
pwd
```

You should see your original location, then a new path ending in `git-course` after the second `pwd`. That confirms `cd` actually moved you.

7. Common Mistakes

COMMON MISTAKE

Typing `cd Git-Course` when the folder is named `git-course`. On macOS/Linux, file names are case-sensitive — capitalization matters.

COMMON MISTAKE

Forgetting which folder you're in and running commands in the wrong place. Get in the habit of running `pwd` whenever you're unsure.

Check Your Understanding

Answer these before moving on:

- 1 What's the difference between an absolute path and a relative path?
- 2 Which command shows you your current location?
- 3 In your own words, why does Git require terminal comfort before anything else?

STOP READING

Open a terminal right now. Create a folder called `git-course` somewhere you'll remember (like your Desktop or home folder). You'll use it starting in Lesson 1.

LEVEL 1 | Foundations

The most basic concepts — what Git is and how to start using it

Lesson 1: What Git Is & Why Version Control Exists

BEGINNER

1. What You Will Learn

- What "version control" means and what problem it solves
- What Git specifically is, and how it differs from GitHub
- Why almost every professional software team uses it

2. Why This Matters

Every programmer eventually breaks something and wishes they could go back in time. Every team eventually has two people edit the same file and needs a way to combine their work without one person's changes erasing the other's. Git exists to solve exactly these two problems — and once you understand it, you'll wonder how anyone codes without it.

3. The Simple Explanation

Imagine writing an essay in a word processor, and every single time you save, the program keeps the *old* version too, instead of overwriting it — labeled with what changed and why. Later, you could open any past version, compare two versions side by side, or combine edits from two different people who worked on the essay separately. That's version control. **Git** is a specific, extremely popular tool that does this for code (and any text files).

THINK ABOUT THIS

Before Git existed, some teams literally kept files named `essay_final.docx`, `essay_final_v2.docx`, `essay_FINAL_ACTUAL.docx`. Git replaces that entire mess with one organized system.

4. Key Terms

Version control

Simple definition: A system for recording changes to files over time.

In plain English: It lets you see history, undo mistakes, and combine work from multiple people.

Example: Google Docs' "version history" is a very simple, limited example of the same idea.

Git

Simple definition: A specific version-control program that runs on your computer.

In plain English: Git is the tool. It works entirely offline — you don't need the internet to use it.

Example: You install Git once, then every project you track with it gets its own private history.

Repository ("repo")

Simple definition: A project folder that Git is tracking the history of.

In plain English: Not every folder is a repo — you have to explicitly tell Git to start tracking one.

Example: Turning a folder into a repo is done with a single command you'll learn in Lesson 3.

Commit

Simple definition: A saved snapshot of your project at a specific point in time.

In plain English: Think of it as a checkpoint you can always return to, with a note explaining what changed.

Example: "Fixed the login bug" might be the message attached to one commit.

5. How It Works

Git and GitHub are **not the same thing** — this trips up almost every beginner.

	Git	GitHub
What it is	A program installed on your computer	A website that hosts Git repositories online
Where it runs	Locally, offline	In the cloud
Analogy	Microsoft Word (the tool)	Google Drive (where you store/share the file)

You can use Git without ever touching GitHub. GitHub becomes useful once you want a backup in the cloud or want to collaborate with other people — covered in Lesson 6.

6. Real-World Example

Every major tech company — Google, Netflix, Anthropic — uses Git internally. Open-source projects like Linux, Python, and React are managed with Git. When you see a job posting requiring "Git experience," this is what it means.

7. Practical Example

There's no command to run yet — this lesson is conceptual. Lesson 2 gets your hands on the keyboard.

8. Common Mistakes

COMMON MISTAKE

Saying "push it to Git" when you mean "push it to GitHub." Git is the tool; GitHub is one possible remote destination for your project (there are others, like GitLab and Bitbucket).

9. "What If?" — Variations

- What if you never make a commit? Then Git has nothing to track yet — it only remembers what you explicitly save.

- What if you work completely alone, forever? Git is still useful, purely for the ability to undo mistakes and see history.
- What if two people edit the same line of the same file? Git will flag a conflict for a human to resolve — covered in Lesson 5.

Check Your Understanding

- 1 In your own words, what problem does version control solve?
- 2 What is the difference between Git and GitHub?
- 3 What is a commit, roughly?
- 4 Why might a solo developer still want to use Git?

Mini Challenge

Without looking back at the lesson, explain to an imaginary teammate why your team should use Git instead of emailing zip files of the project back and forth.

Lesson Summary

- Version control tracks changes to files over time and lets you undo or compare them.
- Git is the local tool; GitHub is a cloud host for Git repositories — they are different things.
- A repository is a folder Git is tracking; a commit is a saved snapshot with a message.

13. Can You Explain It?

Try explaining this concept in your own words, out loud or in writing, without looking at the lesson:

- 1 "Version control is like ____ because ____."
- 2 Explain the Git vs. GitHub difference to a non-programmer friend.
- 3 Why does a checkpoint (commit) need a message attached to it?

Lesson 2: Installing & Configuring Git

BEGINNER

1. What You Will Learn

- How to check whether Git is already installed
- How to install Git if it isn't
- How to set your identity (name and email) so commits are correctly labeled

2. Why This Matters

Every commit you make gets stamped with an author name and email. If this isn't set, Git will refuse to commit or will label your work "unknown," which becomes confusing on any project with more than one contributor.

3. The Simple Explanation

You configure Git exactly once per computer (not once per project). It's like writing your name on the inside cover of a notebook before you start using it.

4. Key Terms

Global config

Simple definition: Settings that apply to every Git repository on your computer.

In plain English: Set once, used everywhere, unless a specific project overrides it.

Example: Your name and email are almost always set globally.

5. How It Works

Step 1 — Check if Git is installed

```
git --version
```

If you see a version number like `git version 2.42.0`, you're set. If you see "command not found," you need to install it.

Step 2 — Install (if needed)

- **macOS:** install Xcode Command Line Tools (`xcode-select --install`) or use Homebrew: `brew install git`
- **Windows:** download and run the installer from git-scm.com — this also installs Git Bash
- **Linux (Debian/Ubuntu):** `sudo apt install git`

Step 3 — Set your identity

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Step 4 — Verify

```
git config --list
```

6. Real-World Example

This is the exact setup step every new hire does on their first day when they get a work laptop.

7. Practical Example

Run the four commands above right now, using your own name and email.

8. Common Mistakes

COMMON MISTAKE

Forgetting the `--global` flag and wondering why the setting "disappeared" in a different project — without it, the setting only applies to whichever repo you're currently in.

COMMON MISTAKE

Using a fake or placeholder email. If you ever connect to GitHub, your commit email should generally match your GitHub account so commits are correctly attributed to you.

Check Your Understanding

- 1 What command checks your installed Git version?
- 2 What's the difference between a global and a local (per-repository) config setting?
- 3 Why does Git need your name and email before you can commit?

Lesson Summary

- Install Git once per machine; verify with ``git --version``.
- Set identity once with ``git config --global user.name/user.email``.
- ``git config --list`` shows your current settings.

Lesson 3: The Core Workflow — init, add, commit, status, log

BEGINNER

1. What You Will Learn

- The three-stage model: working directory → staging area → repository
- How to turn a folder into a Git repository
- How to save your first commits and read the project history

2. Why This Matters

This lesson is the beating heart of the entire course. Nearly everything else — branching, merging, remotes — is a variation on this same core loop. If you deeply understand this lesson, the rest of the course will feel like extensions of something familiar rather than brand-new material.

3. The Simple Explanation

Git doesn't save every change automatically the moment you edit a file — that would be chaotic. Instead, saving happens in two deliberate steps, like packing a box and then mailing it:

- 1 You **stage** the changes you want to save — like putting specific items into a box (this is ``git add``).
- 2 You **commit** the box — sealing it with a label describing what's inside (this is ``git commit``).

This two-step process means you can edit five files but choose to commit only two of them together, keeping your history organized and meaningful.

4. Key Terms

Working directory

Simple definition: The actual files on disk that you see and edit in your editor.

In plain English: This is the messy, in-progress version of your project.

Example: Opening a file in VS Code and typing edits into it happens in the working directory.

Staging area ("the index")

Simple definition: A holding area for changes you're about to commit.

In plain English: It's the "box" you're packing before sealing it — a preview of your next commit.

Example: Running ``git add file.py`` moves that file's current changes into the staging area.

Repository / commit history

Simple definition: The permanent, saved record of every commit you've made.

In plain English: Once something is committed, it's saved in history and can always be found again.

Example: ``git log`` shows you this history.

git init

Simple definition: The command that turns an ordinary folder into a Git repository.

In plain English: It creates a hidden ``.git`` folder that stores all of Git's tracking data.

Example: Run once, inside the folder you want to track: ``git init``.

5. How It Works — Step by Step

The core loop you will use constantly

```
git init                # Turn this folder into a Git repository
git status              # See what's changed / what's staged
git add filename.txt    # Stage a specific file
git add .               # Stage everything that's changed
git commit -m "message" # Save a snapshot with a description
git log                 # View commit history
git log --oneline        # View a compact, one-line-per-commit history
```

The Three Trees, Visually

```
[ Working Directory ] --git add--> [ Staging Area ] --git commit--> [ Repository ]
    (your edits)                   (what's "packed")                 (permanent history)
```

6. Real-World Example

A backend developer fixing a bug edits three files, but only two of the changes are related to the bug fix — the third is an unrelated formatting cleanup. They `git add` just the two related files and commit them together with a focused message, keeping the formatting change for a separate commit. This is exactly the kind of intentional history Git's staging step enables.

7. Practical Example — Do This Now

STOP READING

In your `git-course` folder from Lesson 0, follow along with every command below.

```
cd git-course
git init
echo "My Git Learning Journal" > notes.txt
git status
```

`git status` should show `notes.txt` as an untracked file. Now stage and commit it:

```
git add notes.txt
git status
git commit -m "Add initial notes file"
git log
```

Now make a second change and repeat the cycle:

```
echo "Day 2: learned about staging" >> notes.txt
git status
git add notes.txt
git commit -m "Add day 2 journal entry"
git log --oneline
```

You should now see two commits listed, newest first.

8. Common Mistakes

COMMON MISTAKE

Running ``git commit -m "message"`` without ever running ``git add`` first, then wondering why nothing was saved — nothing gets committed if nothing was staged (unless you use ``git commit -am``, which only works for already-tracked files).

COMMON MISTAKE

Writing vague commit messages like "stuff" or "fix". A good commit message explains *why*, not just *what* — future-you will thank present-you.

COMMON MISTAKE

Forgetting ``git status`` exists. It is the single most useful command in Git for beginners — run it constantly to know exactly what state you're in.

9. "What If?" Variations

- What if you edit a file after staging it, but before committing? Git will show it as both staged and modified — you'd need to ``git add`` again to include the newest edit.
- What if you run ``git commit`` with no ``-m``? Git opens a text editor for you to write a longer message — press ``:wq`` in vim or close the editor after typing to continue.
- What if you ``git init`` inside a folder that's already a Git repo? Git will simply say it reinitialized an existing repository — usually harmless, but avoid nesting repos inside each other.

Check Your Understanding

- 1 What are the three "trees" (locations) a change passes through in Git?
- 2 What does ``git add`` actually do?
- 3 Why might a developer stage only some of their changed files instead of all of them?
- 4 What's the difference between ``git log`` and ``git log --oneline``?
- 5 In your own words, why does Git separate "staging" from "committing" instead of just saving directly?

Mini Challenge

Create a new file called `todo.txt` with three lines of text, each on its own line, added as three separate commits (edit, stage, commit — three times). Then run `git log --oneline` and confirm you see three distinct commits.

Lesson Summary

- Changes flow: working directory → staged (git add) → committed (git commit).
- `git init` starts tracking a folder; `git status` shows current state; `git log` shows history.
- Committing in focused, well-described snapshots keeps your project history genuinely useful.

13. Can You Explain It?

- 1 Explain the working directory / staging area / repository model using a real-world packing analogy of your own.
- 2 Why does `git status` matter so much to a beginner?
- 3 Describe, without naming the commands, what happens when you "save progress" in Git.

Review Session 1

Mixes concepts from Lessons 0–3. Don't skip this — spaced review is what moves facts from short-term to long-term memory.

EASY

- 1 What command shows your current terminal location?
- 2 What is a commit?
- 3 What command turns a folder into a Git repository?

MEDIUM

- 1 Why are Git and GitHub different things?
- 2 Walk through what happens, step by step, when you run ``git add`` then ``git commit``.
- 3 Why does Git need your name and email configured before your first commit?

HARD

- 1 A file has been edited, staged, then edited again. What will ``git status`` show, and why?
- 2 Explain why separating "staging" from "committing" gives you more control than saving directly.

Review Session 1 — Answer Key

1. ``pwd`` (print working directory).
2. A saved snapshot of your project's files at a specific moment, with a message describing the change.
3. ``git init``.
4. Git is the local, offline tool that tracks history; GitHub is a website that hosts Git repositories in the cloud and adds collaboration features. You can use Git fully without ever using GitHub.
5. ``git add`` copies the current state of the specified file(s) into the staging area (a preview of the next commit). ``git commit`` takes everything currently staged and permanently saves it to the repository's history with your message attached.
6. Because every commit is stamped with an author. Without a configured name/email, Git either refuses to commit or records the author as unknown — which is unusable on any team project and confusing even for solo history review.
7. It will show the file as having both staged changes (the first edit) and unstaged changes (the second edit) — because staging captured a snapshot at the moment of ``git add``, and anything edited afterward isn't included until staged again.
8. Staging lets you review exactly what will be committed before it's permanent, and lets you commit only some of your changed files at a time — producing focused, meaningful commits instead of one giant commit that mixes unrelated work.

What Should I Still Remember?

- The three-tree model: working directory → staging → repository, and the commands that move data between them.

- That ``git status`` and ``git log`` are your two main "what's going on" commands.
- That Git and GitHub are not interchangeable words.

Knowledge Checkpoint 1

Before moving to branching, make sure Levels 0–1 are solid — branching builds directly on the commit model.

Self-check statement	Ready / Review / Not Ready
I can explain what a commit is without notes.	_____
I can run <code>init</code> → <code>add</code> → <code>commit</code> → <code>log</code> from memory.	_____
I understand why staging is a separate step from committing.	_____
I know the difference between Git and GitHub.	_____

THINK ABOUT THIS

If you marked anything red, revisit Lesson 3 and redo the Practical Example by hand before continuing — everything from here builds on that loop.

LEVEL 2 | Core Concepts

Branching, merging, and working with remote repositories

Lesson 4: Branching

DEVELOPING

1. What You Will Learn

- What a branch actually is under the hood (it's simpler than people expect)
- How to create, switch to, and list branches
- Why branches let multiple lines of work happen without interfering with each other

2. Why This Matters

Without branches, everyone would have to edit the same single timeline of commits — meaning half-finished, experimental, or broken work would sit directly in the main project. Branches let you build a feature or try an idea in isolation, then bring it into the main line only once it's ready.

3. The Simple Explanation

A branch is just a movable label pointing at a specific commit. When you create a new branch, Git doesn't copy your whole project — it just adds a new sticky note pointing at the commit you're currently on. As you commit on that branch, the sticky note moves forward with you, while the original branch's sticky note stays where it was.

THINK ABOUT THIS

This is the part beginners most often overcomplicate. A branch is not a folder, not a copy, not a zip file — it's a lightweight pointer. That's why creating branches in Git is instant, even on huge projects.

4. Key Terms

Branch

Simple definition: A movable pointer to a specific commit, representing an independent line of work.

In plain English: Think of it as a labeled bookmark that moves forward each time you commit while "on" it.

Example: ``main`` is the default branch most projects start with.

HEAD

Simple definition: A pointer to whatever commit/branch you currently have checked out.

In plain English: It answers the question "where am I right now in the project's history?"

Example: If HEAD points to ``main``, you're working on the main branch.

Checkout / switch

Simple definition: The act of moving HEAD to point at a different branch (or commit).

In plain English: This changes the files in your working directory to match that branch's latest commit.

Example: ``git switch feature-x`` moves you onto the ``feature-x`` branch.

5. How It Works

Branch commands

```
git branch                # List branches (current one marked with *)
git branch feature-login  # Create a new branch (doesn't switch to it)
git switch feature-login  # Switch to that branch
git switch -c feature-login # Create AND switch, in one step
git branch -d feature-login # Delete a branch (once merged)
```

Visualizing a Branch

```
main:      A---B---C
           |
feature-login: D---E
```

Commits A, B, C exist on ``main``. When ``feature-login`` was created at commit C, it started its own path — D and E only exist on ``feature-login`` until it's merged back.

6. Real-World Example

A developer assigned to build a login page creates a branch called ``feature-login``, works there for several days making many commits, while ``main`` stays stable and deployable the whole time. Only when the feature is finished and reviewed does it get merged into ``main``.

7. Practical Example

```
cd git-course
git branch                # should show just "main"
git switch -c feature-hello
echo "Hello from a branch" > branch-file.txt
git add branch-file.txt
git commit -m "Add branch-file from feature-hello"
git switch main
ls                        # notice branch-file.txt is GONE here
git switch feature-hello
ls                        # it's back — you're on a different timeline
```

8. Common Mistakes

COMMON MISTAKE

Editing files while forgetting which branch you're on, and committing work to the wrong branch. Get in the habit of checking ``git branch`` or your terminal prompt before committing.

COMMON MISTAKE

Thinking a new branch "copies" the project, causing unnecessary fear about disk space or complexity. It's just a pointer — creating one is nearly instant and cheap.

9. "What If?" Variations

- What if you delete a branch that hasn't been merged? Git will warn you and refuse with ``-d``; you'd have to force it with ``-D``, which does lose that work.
- What if two branches never interact? They can coexist indefinitely — Git doesn't require branches to ever be merged.
- What if you commit on ``main`` by accident meaning to be on a feature branch? You can move that commit to the correct branch — a fixable, common situation covered in the Problem-Solving Lab.

Check Your Understanding

- 1 In one sentence, what is a branch, technically?
- 2 What does HEAD point to?
- 3 What's the difference between ``git branch feature-x`` and ``git switch -c feature-x``?
- 4 Why are branches cheap to create in Git?

Mini Challenge

Create two new branches from ``main`` called ``experiment-a`` and ``experiment-b``. On each one, commit a different one-line text file. Switch between them and confirm the files appear and disappear as expected.

Lesson Summary

- A branch is a lightweight, movable pointer to a commit — not a copy of the project.
- HEAD tracks which branch/commit you currently have checked out.
- ``git switch -c name`` creates and moves to a new branch in one step.

Lesson 5: Merging & Merge Conflicts

DEVELOPING

1. What You Will Learn

- How to bring one branch's work into another with ``git merge``
- What a fast-forward merge is versus a merge commit
- How to recognize, read, and resolve a merge conflict without panicking

2. Why This Matters

Branching is only half the story — eventually the work has to come back together. Merge conflicts are the single most feared moment for Git beginners, almost always because nobody explained what the conflict markers actually mean. This lesson removes that fear.

3. The Simple Explanation

Merging takes the changes from one branch and combines them into another. Most of the time, Git can do this automatically because the changes touch different parts of the file. A **conflict** only happens when both branches changed the *exact same lines* — Git isn't smart enough to guess which version you want, so it pauses and asks you to decide.

4. Key Terms

Fast-forward merge

Simple definition: A merge where the target branch simply moves its pointer forward — no new commit needed.

In plain English: This happens when the branch being merged into hasn't changed at all since the other branch split off.

Example: Merging ``feature-x`` into ``main`` fast-forwards if nobody committed to ``main`` in the meantime.

Merge commit

Simple definition: A special commit with two parents, created when histories have diverged and need combining.

In plain English: It's the "stitching point" where two separate timelines become one again.

Example: You'll see this in ``git log`` as a commit joining two branch lines.

Merge conflict

Simple definition: A situation where Git cannot automatically decide how to combine two changes to the same lines.

In plain English: Git pauses the merge and inserts special markers into the file showing both versions for you to choose between.

Example: Covered fully below with a real example.

5. How It Works

Basic merge

```
git switch main
git merge feature-login    # Bring feature-login's commits into main
```

Reading Conflict Markers

When a conflict happens, Git edits the file to show both versions, separated by markers:

```
<<<<<< HEAD
This line came from main.
=====
This line came from feature-login.
>>>>>> feature-login
```

You manually edit the file to keep whichever content is correct (possibly a blend of both), delete the marker lines entirely, then stage and commit the result.

```
# after manually editing the file to resolve it:
git add conflicted-file.txt
git commit -m "Resolve merge conflict between main and feature-login"
```

6. Real-World Example

Two developers both edit the introduction paragraph of a README file on different branches. When merged, Git can't guess which intro to keep — a human reads both versions and decides (often by combining the best of each) before completing the merge.

7. Practical Example — Create a Real Conflict on Purpose

```
git switch main
echo "Line from main" > shared.txt
git add shared.txt
git commit -m "Add shared.txt on main"

git switch -c conflict-branch
echo "Line from conflict-branch" > shared.txt
git add shared.txt
git commit -m "Edit shared.txt on conflict-branch"

git switch main
echo "A DIFFERENT edit on main" > shared.txt
git add shared.txt
git commit -m "Edit shared.txt differently on main"

git merge conflict-branch    # This WILL conflict – that's the point
```

Open `shared.txt` in your editor, resolve the markers by choosing/combining content, then:

```
git add shared.txt
git commit -m "Resolve conflict in shared.txt"
```

8. Common Mistakes

COMMON MISTAKE

Panicking and running ``git merge --abort`` repeatedly instead of just reading the conflict markers — which is safe to use if you genuinely want to cancel, but doesn't teach you to resolve conflicts.

COMMON MISTAKE

Forgetting to delete the ``<<<<<<<``, ``=====``, and ``>>>>>>>`` marker lines themselves before committing — leaving broken marker text in the final file.

COMMON MISTAKE

Resolving a conflict but forgetting to ``git add`` the file afterward — Git won't know the conflict is resolved until you explicitly stage it.

9. "What If?" Variations

- What if the same file is edited but on different lines? Git merges automatically — no conflict, no manual step needed.
- What if you want to cancel a conflicted merge entirely? ``git merge --abort`` returns you to the state before the merge started.
- What if a conflict happens in three files at once? Resolve them one at a time, ``git add`` each as you finish, then commit once all are resolved.

Check Your Understanding

- 1 What's the difference between a fast-forward merge and a merge commit?
- 2 What specifically causes a merge conflict (be precise — it's not just "two people edited the same file")?
- 3 What do the ``<<<<<<<``, ``=====``, and ``>>>>>>>`` markers each represent?
- 4 What are the exact steps to resolve a conflict once you've edited the file?

Mini Challenge

Deliberately create another conflict (different file, different content) and resolve it without looking back at the Practical Example above.

Lesson Summary

- ``git merge branch-name`` combines another branch's commits into your current branch.
- Fast-forward merges just move a pointer; diverged histories need a merge commit.
- A conflict means Git needs a human decision — edit out the markers, choose content, add, commit.

Lesson 6: Remotes — clone, push, pull, fetch

DEVELOPING

1. What You Will Learn

- What a "remote" repository is and how it relates to your local one
- The difference between push, pull, and fetch
- How to connect a local repo to GitHub and sync work both directions

2. Why This Matters

Everything so far has lived only on your computer. Remotes are how your work gets backed up, shared with teammates, and deployed. This is also the point where Git stops being a personal notebook and becomes a collaboration tool.

3. The Simple Explanation

A **remote** is just another copy of the repository, usually hosted somewhere online like GitHub, that your local repository knows how to talk to. "Pushing" sends your commits there; "pulling" brings their commits to you.

4. Key Terms

Remote

Simple definition: A version of your repository hosted elsewhere (commonly on GitHub) that your local repo can sync with.

In plain English: By default it's named ``origin`` — that's just a conventional nickname, not a special keyword.

Example: ``git remote -v`` lists the remotes your local repo knows about.

Clone

Simple definition: Downloading a full copy of an existing remote repository, including its entire history.

In plain English: This is how you "start" working on a project that already exists on GitHub.

Example: ``git clone https://github.com/user/repo.git``

Push

Simple definition: Sending your local commits to a remote repository.

In plain English: This uploads your saved history so others (or your future self) can see it.

Example: ``git push origin main`` sends your ``main`` branch's new commits to ``origin``.

Pull

Simple definition: Downloading and merging a remote's new commits into your current local branch.

In plain English: It's fetch + merge combined into one step.

Example: ``git pull origin main`` updates your local ``main`` with any new commits from the remote.

Fetch

Simple definition: Downloading a remote's new commits without merging them into your current work.

In plain English: This lets you look at what changed before deciding to bring it in.

Example: ``git fetch origin`` updates your knowledge of the remote without touching your files.

5. How It Works

```
git clone https://github.com/user/repo.git    # Download an existing project
git remote -v                                # See what remotes are configured
git remote add origin https://github.com/you/repo.git  # Connect a NEW local repo to
git push -u origin main                       # First push (sets up tracking)
git push                                     # Later pushes (tracking already set)
git pull                                     # Get + merge the latest remote changes
git fetch                                    # Get the latest info WITHOUT merging
```

6. Real-World Example

A developer starts their day by pulling the latest changes their teammates pushed overnight, does their own work in a feature branch, then pushes that branch to GitHub so a teammate can review it before it's merged into `main`.

7. Practical Example

TIP

This exercise assumes you have a free GitHub account. If you don't have one yet, create one at github.com before continuing — it takes a couple of minutes.

- 1 On GitHub, create a new empty repository (no README) named `git-course-remote`.
- 2 Copy the HTTPS URL GitHub shows you.

```
cd git-course
git remote add origin <paste-the-URL-here>
git branch -M main
git push -u origin main
```

Refresh the GitHub page — your commits should now appear online. Now simulate a teammate's change by editing the file directly on GitHub's website, then pull it locally:

```
git pull
```

8. Common Mistakes

COMMON MISTAKE

Trying to push before ever pulling on a repo someone else has also pushed to — Git will reject the push because histories have diverged. Pull first, resolve any conflicts, then push.

COMMON MISTAKE

Confusing `pull` and `fetch`. `fetch` is the safe "just look" option; `pull` immediately merges, which can create a conflict if you weren't expecting one.

COMMON MISTAKE

Forgetting `-u` on the very first push, then having to type the full remote and branch name every time afterward instead of a plain `git push`.

9. "What If?" Variations

- What if your push is rejected? Run `git pull` first to merge in the remote's new commits, resolve any conflicts, then push again.
- What if you clone a repo instead of connecting an existing folder to one? Cloning already sets up the `origin` remote for you automatically.
- What if you want to see changes before merging them? Use `git fetch`, then `git log origin/main` to inspect before deciding to `git merge origin/main`.

Check Your Understanding

- 1 What is a remote, in your own words?
- 2 What's the practical difference between `git pull` and `git fetch`?
- 3 Why might `git push` get rejected, and what's the fix?
- 4 What does `git clone` do that `git init` does not?

Mini Challenge

Make one more commit locally, push it, then edit a different line directly on GitHub's website and pull that change down. Confirm both changes now exist in your local `git log`.

Lesson Summary

- A remote is another copy of your repo, usually on GitHub, referenced by a nickname like `origin`.
- Push sends your commits out; pull brings + merges remote commits in; fetch only downloads info.
- Pushed-then-rejected almost always means: pull first, resolve, then push again.

Mini Project 1: Personal Notes Repo

GOAL

Combine init, add, commit, branch, and merge into one small realistic project — no new commands, just putting the pieces together.

Objective

Build a small personal-notes project tracked with Git, using a feature branch for a new section and merging it back in.

Requirements

- A repository called `personal-notes`` with at least 4 commits on `main``
- One feature branch that adds a new file, merged back into `main``
- A readable commit history (check with `git log --oneline``)

Step-by-Step Guidance

- 1 `git init personal-notes`` and `cd`` into it.
- 2 Create `ideas.txt`` with one idea per line, committing after each addition (3+ commits).
- 3 Create a branch called `add-goals``, add a `goals.txt`` file there, commit it.
- 4 Switch back to `main`` and merge `add-goals`` in.
- 5 Confirm with `git log --oneline --graph --all`` that the merge is visible in history.

TIP

`git log --oneline --graph --all`` draws a simple text diagram of your branch and merge history — an extremely useful command to reach for whenever you're unsure what actually happened.

Common Mistakes To Watch For

- Committing directly to `main`` when you meant to be on `add-goals`` — check `git branch`` first.
- Forgetting to `git add`` before `git commit`` and getting a confusing "nothing to commit" message.

Testing Checklist

- [] `git log --oneline`` shows 4+ commits
- [] `git branch -a`` shows both `main`` and `add-goals`` (or confirms `add-goals`` was deleted after merging)
- [] `goals.txt`` exists in `main`` after the merge

Expected Result

A single repository where ``main`` contains everything from both the direct commits and the merged feature branch, with a clean, readable history.

Extension Challenge

Redo this project, but this time deliberately edit the same line of ``ideas.txt`` on both ``main`` and a second branch before merging, so you have to resolve a real conflict.

Review Session 2

Mixing Levels 0 through 2 — old material stays in the mix on purpose.

EASY

- 1 What command lists your branches?
- 2 What's the default name Git gives a remote when you clone a repo?
- 3 What does ``git status`` tell you?

MEDIUM

- 1 Explain the difference between ``git merge`` and ``git pull`` — how are they related?
- 2 Why is a branch described as a "pointer" rather than a "copy"?
- 3 What causes a merge conflict, precisely?

HARD

- 1 You and a teammate both pushed commits, but yours was rejected. Walk through exactly what you'd do next, in order.
- 2 Explain, using the working-directory/staging/repository model from Lesson 3, what state a file is in immediately after a merge conflict is created.

Review Session 2 — Answer Key

1. ``git branch``.

2. ``origin``.

3. It shows the current state of your working directory and staging area — what's changed, what's staged, and what branch you're on.

4. ``git pull`` is actually ``git fetch`` (download remote commits) followed automatically by ``git merge`` (combine them into your current branch) — pull is a two-step shortcut.

5. Because creating one doesn't duplicate any files or history — it just adds a new movable label at your current commit, which is why it's instant even on huge repositories.

6. A merge conflict happens specifically when two branches both changed the exact same lines of the exact same file — if the changes touch different lines or different files, Git merges them automatically without asking.

7. Run ``git pull`` to bring down and merge the teammate's commits (resolving any conflict that results), confirm the merge looks correct, then run ``git push`` again — it should now succeed since your local branch includes their work.

8. The conflicting file sits in the working directory in a modified, unresolved state containing conflict markers; it is also marked by Git as "unmerged" — it is not staged and cannot be committed until you edit out the markers and run ``git add`` on it.

What Should I Still Remember?

- Branches are pointers, not copies — cheap to create, easy to switch between.

- Merge conflicts only happen on overlapping line changes, and are resolved by hand-editing conflict markers.
- Push/pull/fetch each do a different, specific thing — don't use them interchangeably in your head.

Knowledge Checkpoint 2

Self-check statement	Ready / Review / Not Ready
I can create, switch to, and merge a branch without looking anything up.	_____
I can resolve a merge conflict by reading the markers.	_____
I understand push vs. pull vs. fetch and when to use each.	_____
I completed Mini Project 1 successfully.	_____

THINK ABOUT THIS

Red marks on the conflict-resolution row is extremely common at this stage. Redo the Practical Example in Lesson 5 by hand — it's a skill that clicks with repetition, not re-reading.

LEVEL 3 | Practical Usage

How Git is actually used day to day on real projects

Lesson 7: The Everyday Workflow — diff, .gitignore, stash, feature branches

INTERMEDIATE

1. What You Will Learn

- How to review exactly what changed before committing, using ``git diff``
- How to tell Git to permanently ignore certain files with ``.gitignore``
- How to temporarily set aside unfinished work with ``git stash``
- The feature-branch pattern real teams use for nearly every change

2. Why This Matters

Lessons 1–6 gave you the mechanics. This lesson gives you the habits professional developers actually run dozens of times a day. Skipping these tools doesn't break Git — it just makes using it noticeably more painful.

3. The Simple Explanation

``git diff`` lets you proofread your changes before saving them, ``.gitignore`` keeps junk files (like logs or build output) out of your history entirely, and ``git stash`` is like a pause button for unfinished work when something more urgent comes up. Feature branches are simply the habit of using Lesson 4's branches for every piece of work, not just big features.

4. Key Terms

.gitignore

Simple definition: A file listing patterns of files/folders Git should never track.

In plain English: It stops secrets, build artifacts, and machine-specific junk from ever entering your history.

Example: A Python project's ``.gitignore`` typically includes ``__pycache__/'` and ``.env``.

git diff

Simple definition: Shows the exact line-by-line changes between your working directory and what's staged/committed.

In plain English: Read it before every commit — it's your last chance to catch a mistake.

Example: ``git diff`` shows unstaged changes; ``git diff --staged`` shows what's about to be committed.

git stash

Simple definition: Temporarily shelves your uncommitted changes so your working directory becomes clean again.

In plain English: It's a scratch space for "I need to switch tasks right now but I'm not ready to commit this."

Example: ``git stash`` saves and clears; ``git stash pop`` brings it back.

5. How It Works

```
git diff                # See unstaged changes, line by line
git diff --staged       # See what's staged and about to be committed

echo "*.log" >> .gitignore
echo ".env" >> .gitignore
git add .gitignore
git commit -m "Add .gitignore for logs and env files"

git stash               # Shelve current uncommitted work
git stash list          # See stashed items
git stash pop           # Restore the most recent stash
```

The Feature-Branch Pattern

```
git switch main
git pull
git switch -c feature/user-avatars # descriptive branch name, work isolated
# ... make commits ...
git push -u origin feature/user-avatars
# open a Pull Request on GitHub for review, then merge once approved
```

This pattern — branch, work, push, review, merge — is close to what you'll see at nearly every company using Git.

6. Real-World Example

A team's `.gitignore`` typically excludes dependency folders (``node_modules/``), compiled output, IDE settings, and anything containing secrets or credentials. Committing a `.env`` file with real passwords into a public repo is one of the most common and damaging beginner mistakes — `.gitignore`` prevents it before it happens.

7. Practical Example

```

cd git-course
echo "print('debug')" > scratch.py
git status
git diff          # will show nothing yet – file is untracked, not modified
git add scratch.py
git diff --staged # now shows the addition

echo "*.pyc" >> .gitignore
echo "__pycache__/" >> .gitignore
git add .gitignore
git commit -m "Add Python gitignore rules"

echo "unfinished idea" >> notes.txt
git stash
git status          # notes.txt change is gone from view
git stash pop
git status          # it's back

```

8. Common Mistakes

COMMON MISTAKE

Adding a `.gitignore` rule *after* a file has already been committed — the rule won't retroactively remove it from history; you'd need `git rm --cached filename` as well.

COMMON MISTAKE

Forgetting a stash exists and losing track of unfinished work — `git stash list` should be checked if you ever feel like something "disappeared."

COMMON MISTAKE

Naming feature branches vaguely, like `fix` or `update`, instead of something descriptive like `fix/login-timeout` — this matters a lot once a team has many branches at once.

9. "What If?" Variations

- What if you stash, then switch branches, then pop the stash? It works — stashes aren't tied to the branch you created them on, though popping onto a very different branch can occasionally cause conflicts.
- What if you have multiple stashes at once? `git stash list` shows them all; `git stash pop` always restores the most recent unless you specify one by name.
- What if a file is already tracked and you add it to `.gitignore`? Git will keep tracking it until you explicitly remove it from tracking with `git rm --cached`.

Check Your Understanding

- 1 What's the difference between `git diff` and `git diff --staged`?

- 2 Why can't `.gitignore` remove a file that's already been committed?
- 3 Describe a situation where `git stash` would be the right tool.
- 4 Walk through the feature-branch pattern from start to merge, in your own words.

Mini Challenge

Create a `.gitignore` that excludes a `secrets.txt` file, confirm with `git status` that Git no longer notices it after creating it, then use `git stash` to shelve an unrelated in-progress change and restore it.

Lesson Summary

- `git diff` (and `--staged`) let you proofread changes before committing them.
- `.gitignore` prevents specific files/patterns from ever being tracked — set it up early in a project.
- `git stash` shelves uncommitted work temporarily; the feature-branch pattern is the default real-world workflow.

LEVEL 4 | Problem Solving

Diagnosing and fixing situations you WILL eventually run into

Problem-Solving Lab: Diagnosing Real Git Situations

No new commands here — this level is entirely about applying Levels 1–3 to messy, realistic situations. Try to solve each one yourself before reading the guidance.

Scenario 1: Committed to the wrong branch

Situation: You meant to be on `feature-x` but accidentally committed straight to `main`.

GUIDANCE (try first, then reveal)

Note the commit hash from `git log` (or use `git log -1`), switch to the correct branch (`git switch feature-x`), then `git cherry-pick <hash>` to bring that commit there. Afterward, go back to `main` and use `git reset --hard HEAD~1` to remove it from `main` (covered fully in Lesson 9).

Scenario 2: "fatal: not a git repository"

Situation: You run `git status` and get an error saying you're not inside a Git repository.

GUIDANCE (try first, then reveal)

Run `pwd` to confirm your location — you're very likely in the wrong folder, or above/outside the folder where you ran `git init`. `cd` into the correct project folder and try again.

Scenario 3: Accidentally staged a file you didn't mean to

Situation: You ran `git add .` and it grabbed a file you're not ready to commit.

GUIDANCE (try first, then reveal)

Run `git restore --staged filename` to unstage it without losing your edits — the file stays changed in your working directory, just no longer queued for the next commit.

Scenario 4: Push rejected — "failed to push some refs"

Situation: You try to push and Git refuses, saying the remote contains work you don't have locally.

GUIDANCE (try first, then reveal)

This means someone else pushed since you last pulled. Run `git pull` to merge their commits in (resolving conflicts if any appear), then `git push` again.

Scenario 5: Wrote the wrong commit message

Situation: You committed with a typo or unclear message and haven't pushed yet.

GUIDANCE (try first, then reveal)

Run ``git commit --amend -m "corrected message"`` to rewrite the most recent commit's message. Never amend a commit you've already pushed and shared with others.

Check Your Understanding

- 1 Which command unstages a file without discarding its changes?
- 2 Why does ``git push`` sometimes get rejected, and what's the standard fix?
- 3 What's the safety rule around ``git commit --amend``?

LEVEL 5 | Intermediate Concepts

More powerful, more careful tools built on the foundations

Lesson 8: Rebasing vs Merging

INTERMEDIATE

1. What You Will Learn

- What ``git rebase`` does differently from ``git merge``
- Why rebasing produces a cleaner, linear history
- When to prefer merge over rebase, and the golden rule of rebasing

2. Why This Matters

This is one of the harder concepts in this course. It's worth slowing down here — rebasing is powerful but has one rule that, if broken, causes real pain for a team. Understanding it properly now avoids that pain later.

IMPORTANT

This is one of the harder concepts in this subject. We'll break it into small pieces below rather than rushing through it.

3. The Simple Explanation

Both ``merge`` and ``rebase`` combine work from two branches — but they tell the story differently. **Merging** preserves exactly what happened: both timelines stay visible, joined by a merge commit. **Rebasing** rewrites history to look as if your branch's commits happened one-by-one, freshly, right after the latest commit on the branch you're rebasing onto — producing a single straight line instead of a branching tree.

4. Key Terms

Rebase

Simple definition: Replaying your branch's commits one at a time on top of another branch's latest commit.

In plain English: It rewrites commit history to look linear, as though your work happened after theirs, sequentially.

Example: ``git rebase main`` while on ``feature-x`` replays feature-x's commits on top of main's current tip.

Linear history

Simple definition: A commit history with no branching/merging structure — one straight line.

In plain English: Easier to read in ``git log``, at the cost of no longer showing exactly when branches diverged.

Example: Many teams prefer this for a clean ``main`` branch history.

5. How It Works

```
# Merge approach (preserves history exactly as it happened):
git switch main
git merge feature-x          # creates a merge commit joining both timelines

# Rebasing approach (rewrites feature-x's commits to sit on top of main):
git switch feature-x
git rebase main              # feature-x's commits are replayed after main's latest c
git switch main
git merge feature-x          # now fast-forwards cleanly — no merge commit needed
```

Visual Comparison

```
MERGE:
main:    A---B---C-----M
          \         /
feature:  D---E---/

REBASE:
main:    A---B---C
feature:  D'---E'   (D and E rewritten as D' and E', replayed after C)
```

6. Real-World Example

Many teams use rebase to keep an individual feature branch up to date with `main` day-to-day (so it stays a clean, easy-to-review line of commits), but use a regular merge (often via a Pull Request) for the final step of bringing a finished feature into `main`.

COMMON MISTAKE

THE GOLDEN RULE: Never rebase commits that have already been pushed and that other people might have based work on. Rebasing rewrites commit history — anyone who already pulled the old version will get conflicting, duplicated history. Only rebase your own local, unpushed (or solely-yours) work.

7. Practical Example

```
git switch -c rebase-practice
echo "first change" > rebase-file.txt
git add rebase-file.txt
git commit -m "First change on rebase-practice"

git switch main
echo "unrelated main change" > main-file.txt
git add main-file.txt
git commit -m "Unrelated change on main"

git switch rebase-practice
git rebase main
git log --oneline --graph --all
```

Notice the history is now a single straight line instead of two diverging paths.

8. Common Mistakes

COMMON MISTAKE

Rebasing a branch that a teammate has already pulled and built work on top of — this is the single most common way rebasing causes real team problems.

COMMON MISTAKE

Panicking mid-rebase if a conflict appears. Just like a merge conflict: edit the file, remove the markers, `git add` it, then run `git rebase --continue`.

9. "What If?" Variations

- What if a conflict happens during rebase? Resolve it like a merge conflict, then run `git rebase --continue` instead of a normal commit.
- What if you want to cancel a rebase halfway through? `git rebase --abort` returns you to exactly where you started.
- What if you're working completely solo? Rebasing is very safe in that case, since there's no one else's history to conflict with.

Check Your Understanding

- 1 In your own words, what does rebase do differently from merge?
- 2 What is the golden rule of rebasing, and why does it exist?
- 3 What command continues a rebase after resolving a conflict?
- 4 When might a team prefer merge over rebase for the final step of combining a feature?

Mini Challenge

Repeat the Practical Example, but this time cause a real conflict during the rebase (edit the same line in `rebase-file.txt` on both branches) and resolve it using `git rebase --continue`.

Lesson Summary

- Merge preserves exact history with a merge commit; rebase rewrites history into a straight line.
- Never rebase commits others have already pulled and built on.
- Conflicts during rebase are resolved the same way as merge conflicts, then `git rebase --continue`.

Lesson 9: Undoing Things — reset, revert, checkout, reflog

INTERMEDIATE

1. What You Will Learn

- Three different ways to "undo" in Git, and when each is appropriate
- The difference between undoing something privately vs. undoing something already shared
- How `git reflog` acts as a safety net for almost any mistake

2. Why This Matters

Undoing is where beginners get genuinely scared of Git — and where experienced developers feel calm, because they know Git almost never actually deletes anything permanently. This lesson is designed to replace fear with a clear decision process.

3. The Simple Explanation

Think of three different "undo" situations: undoing something on your own private branch (nobody's seen it yet), undoing something that's already been shared with others, and undoing an accidental discard of a file's edits. Git has a distinct, correct tool for each.

4. Key Terms

`git restore`

Simple definition: Discards uncommitted changes in your working directory or unstages a file.

In plain English: Use this for "I haven't committed yet, just get rid of this edit."

Example: `git restore filename.txt` reverts it to its last committed state.

`git reset`

Simple definition: Moves your branch pointer backward, optionally undoing commits and/or staging.

In plain English: Rewrites history — safe on private, unpushed work; risky on shared/pushed work.

Example: `git reset --hard HEAD~1` throws away the last commit and all its changes entirely.

`git revert`

Simple definition: Creates a brand NEW commit that undoes the changes of a previous commit.

In plain English: Safe on shared history because it adds to history instead of rewriting it.

Example: `git revert <commit-hash>` is the standard way to undo something already pushed.

`git reflog`

Simple definition: A log of every place HEAD has pointed to recently, including "lost" commits.

In plain English: It's Git's safety net — almost nothing is truly gone until Git's garbage collection runs, much later.

Example: `git reflog` can help you recover a commit even after a `git reset --hard`.

5. How It Works

Situation	Tool	Why
Discard an uncommitted edit	<code>`git restore file`</code>	Nothing was committed yet — just revert the file
Undo commits nobody has seen	<code>`git reset`</code>	Safe to rewrite history that's only local
Undo a commit already pushed/shared	<code>`git revert`</code>	Adds a new undo-commit instead of rewriting shared history
"I think I lost work"	<code>`git reflog`</code>	Shows recent HEAD positions so you can recover

```
git restore filename.txt      # discard uncommitted edits to one file
git restore --staged filename.txt # unstage without discarding edits

git reset --soft HEAD~1      # undo last commit, keep changes staged
git reset --mixed HEAD~1     # undo last commit, keep changes unstaged (default mode)
git reset --hard HEAD~1      # undo last commit AND discard its changes entirely

git revert <commit-hash>     # create a new commit that undoes an earlier one

git reflog                   # see recent HEAD history, including "lost" commits
git reset --hard <hash-from-reflog> # recover to that point
```

6. Real-World Example

A developer realizes a commit pushed to ``main`` yesterday introduced a bug already deployed to production. They can't rewrite that shared history safely, so they ``git revert`` it — adding a clean, new commit that undoes the change, visible to everyone, with no history rewriting required.

7. Practical Example

```
echo "temporary bad line" >> notes.txt
git status
git restore notes.txt
git status                # change is gone, file back to last commit

echo "commit to undo" >> notes.txt
git add notes.txt
git commit -m "Commit I will undo"
git log --oneline -3
git reset --soft HEAD~1   # undo the commit, keep the change staged
git status

git commit -m "Commit I will undo"
git revert HEAD            # undo via a NEW commit instead
git log --oneline -3

git reflog                # see everything that just happened
```

8. Common Mistakes

COMMON MISTAKE

Using ``git reset --hard`` on commits that have already been pushed and shared — this can cause serious confusion for teammates whose local history no longer matches. Use ``revert`` for anything already shared.

COMMON MISTAKE

Panicking after ``git reset --hard`` and assuming the work is gone forever. In almost all cases, ``git reflog`` can find the lost commit's hash so you can recover it.

9. "What If?" Variations

- What if you ``reset --hard`` and immediately regret it? Run ``git reflog``, find the commit hash from before the reset, and run ``git reset --hard``.
- What if you want to undo a commit from three commits ago, not the most recent one? ``git revert`` works on any commit hash, not just the latest.
- What if you just want to see an old version of one file without changing your branch? ``git checkout --filename`` restores just that file from that commit.

Check Your Understanding

- 1 What's the key difference between ``git reset`` and ``git revert``, and when do you use each?
- 2 Why is ``git reset --hard`` risky specifically on pushed/shared commits?
- 3 What does ``git reflog`` show you, and why is it described as a safety net?
- 4 Which tool would you use to simply discard an uncommitted edit to one file?

Mini Challenge

Make a commit, undo it with ``git reset --hard``, then use ``git reflog`` to find and recover it — without looking at the Practical Example above.

Lesson Summary

- ``git restore`` undoes uncommitted changes; ``git reset`` rewrites local history; ``git revert`` safely undoes shared history.
- Never ``reset --hard`` commits others may already have — ``revert`` instead.
- ``git reflog`` is your safety net — very few Git mistakes are actually unrecoverable.

Lesson 10: Tags & Releases

DEVELOPING

1. What You Will Learn

- What a tag is and how it differs from a branch
- How to create and push tags to mark releases
- Basic semantic versioning conventions used alongside tags

2. Why This Matters

Once a project reaches real users, you need a permanent, unmoving marker for "this exact state was version 1.0." Branches move forward; tags don't. That distinction is the whole point of this lesson.

3. The Simple Explanation

A tag is like a branch that never moves again — a permanent bookmark on one specific commit, typically used to mark a release.

4. Key Terms

Tag

Simple definition: A fixed, permanent pointer to one specific commit.

In plain English: Unlike a branch, a tag never advances as new commits are made.

Example: `v1.0.0` might tag the exact commit that was released to users as version 1.0.

Semantic versioning (SemVer)

Simple definition: A version-numbering convention: MAJOR.MINOR.PATCH.

In plain English: MAJOR = breaking changes, MINOR = new features (backward-compatible), PATCH = bug fixes.

Example: Going from `1.2.3` to `1.3.0` signals a new feature was added without breaking existing usage.

5. How It Works

```
git tag v1.0.0           # tag the current commit
git tag v0.9.0 <commit-hash> # tag a specific past commit
git tag                  # list all tags
git push origin v1.0.0    # push one tag to the remote
git push origin --tags    # push all tags at once
```

6. Real-World Example

Open-source projects on GitHub use tags to power their "Releases" page — each release corresponds to one tagged commit, often with release notes attached.

7. Practical Example

```
cd git-course
git tag v1.0.0
git tag
git push origin v1.0.0
```

8. Common Mistakes

COMMON MISTAKE

Forgetting that a plain `git push` does NOT push tags by default — you need `git push origin --tags` or to name the tag explicitly.

Check Your Understanding

- 1 What's the core difference between a tag and a branch?
- 2 In SemVer, what does bumping the MINOR number typically signal?
- 3 How do you push a tag to a remote?

Lesson Summary

- Tags are permanent pointers to a specific commit — typically used to mark releases.
- Tags don't push automatically with `git push` — use `--tags` or name them explicitly.
- SemVer (MAJOR.MINOR.PATCH) is a common convention paired with tags.

Intermediate Project: Simulated Team Collaboration

GOAL

Combine branching, merging, rebasing, remotes, and conflict resolution into one realistic multi-step exercise.

Objective

Simulate two "developers" (both played by you, in two folders) collaborating on the same GitHub repository, including a rebase and an intentional conflict.

Requirements

- One GitHub repo, cloned into two separate local folders (`dev-a` and `dev-b`)
- At least one feature branch rebased onto `main` before merging
- At least one deliberately resolved merge conflict
- A final, clean `git log --oneline --graph` on `main`

Step-by-Step Guidance

- 1 Create a new GitHub repo `team-sim` with a README, then clone it twice: into `dev-a/` and `dev-b/`.
- 2 In `dev-a`, create `feature/header`, add a `header.txt`, commit, push, and merge into `main` via a normal merge.
- 3 In `dev-b`, `git pull` to catch up, then create `feature/footer`, add `footer.txt`, commit.
- 4 Before merging `feature/footer`, run `git rebase main` on it so it sits cleanly on top of the latest `main`, then merge.
- 5 Now in `dev-a`, edit the same line of `header.txt` that `dev-b` also edits in their copy; push from `dev-a`, then try to push from `dev-b` — resolve the resulting conflict.

Hints

- Remember: `git pull` before `git push` whenever the push is rejected.
- Use `git log --oneline --graph --all` in both folders to compare what each "developer" sees.

Common Mistakes

- Rebasing a branch that's already been pushed and pulled by the other "developer" — rebase only before sharing.
- Forgetting which folder (`dev-a` vs `dev-b`) represents which "developer" and getting confused about whose changes are whose.

Testing Checklist

- [] Both `dev-a` and `dev-b` show identical, converged history after the final pull

- [] At least one conflict was created and manually resolved
- [] ``feature/footer`` was rebased before merging (visible as a linear segment in the log)

Expected Result

A single shared GitHub history that both local folders agree on, including evidence of a merge, a rebase, and a resolved conflict.

Extension Challenge

Add a third simulated developer folder (``dev-c``) and have all three push, pull, and resolve conflicts against the same shared ``main``.

Review Session 3

Full-spectrum review — Levels 0 through 5.

EASY

- 1 What's the difference between ``git restore`` and ``git reset``?
- 2 What does a tag point to, and does it move?
- 3 What command shows recent HEAD history, even "lost" commits?

MEDIUM

- 1 Explain, precisely, when you should use ``revert`` instead of ``reset``.
- 2 Why is rebase described as rewriting history, while merge is described as preserving it?
- 3 What is SemVer, and what does each of the three numbers mean?

HARD

- 1 A teammate says "I rebased our shared feature branch and now everyone's history is a mess." Explain exactly what likely went wrong and why.
- 2 Walk through, from memory, the full lifecycle of a single feature: branch creation, commits, rebase, merge, and tag — naming every command involved.

Review Session 3 — Answer Key

1. ``git restore`` discards uncommitted changes in the working directory (or unstages); ``git reset`` moves the branch pointer itself, potentially undoing commits.
2. A tag points to one specific, fixed commit and does not move as new commits are added — unlike a branch.
3. ``git reflog``.
4. Use ``revert`` whenever the commit you're undoing has already been pushed and possibly pulled by others — it adds a new commit rather than rewriting shared history, so nobody's local history breaks.
5. Merge creates a new commit that joins two existing timelines together, leaving the original commits and their order untouched. Rebase takes your branch's commits and replays them as brand-new commits on top of another branch's tip — the originals are effectively replaced, which is why it's called history rewriting.
6. Semantic Versioning is a MAJOR.MINOR.PATCH numbering convention: MAJOR increments for breaking changes, MINOR increments for new, backward-compatible features, and PATCH increments for bug fixes that don't add features or break anything.
7. They likely rebased a branch that other people had already pulled and built commits on top of. Because rebase rewrites the branch's commit hashes, everyone else's local copies now disagree with the new rewritten history, producing duplicate commits and confusing conflicts when they try to sync — this is exactly what the golden rule of rebasing warns against.
8. ``git switch -c feature/x`` (create branch) → edit, ``git add``, ``git commit`` (repeat as needed) → ``git rebase main`` (bring it up to date, before sharing) → ``git switch main``; `git merge feature/x`` (combine it in) → ``git tag v1.x.x`` and ``git push origin --tags`` (mark the release) → ``git push`` (share ``main``).

What Should I Still Remember?

- reset (private/local) vs. revert (shared/pushed) — this distinction matters a lot in real teams.
- The rebase golden rule: never rewrite history others have already pulled.
- Tags mark permanent points in history; branches move forward.

Knowledge Checkpoint 3

Self-check statement	Ready / Review / Not Ready
I know when to use reset vs. revert without hesitating.	_____
I can explain the rebase golden rule to someone else.	_____
I completed the Intermediate Project successfully.	_____
I'm comfortable creating and pushing a tag.	_____

LEVEL 6 | Mini Projects

Smaller combined exercises before the final challenge

Mini Project 2: Fix the Broken History

GOAL

Practice the Level 4 and Level 5 "undo" toolkit on a history that's intentionally messy.

Objective

Deliberately create a messy commit history, then clean it up using the tools from Lessons 8 and 9.

Requirements

- A branch with at least 5 commits, including one with a bad message and one that should be fully discarded
- The bad commit message fixed with `--amend`
- The unwanted commit removed using the appropriate tool for whether it was shared or not

Step-by-Step Guidance

- 1 Create a branch `messy-history` and make 5 commits, including one with the message "asdf" and one that adds a file you don't actually want.
- 2 Amend the "asdf" commit's message to something meaningful (only works cleanly if it's the most recent commit — plan accordingly, or research `rebase -i` as a stretch goal).
- 3 Since this branch hasn't been pushed anywhere, use `git reset` to remove the unwanted commit entirely.
- 4 Confirm the final history with `git log --oneline` reads cleanly, start to finish.

Hints

- `--amend` only rewrites the single most recent commit — order your mistakes accordingly.
- This is unpushed, private history — `reset --hard` is safe to use here.

Testing Checklist

- [] No commit message reads "asdf" or similar placeholder text
- [] The unwanted file no longer appears in `git log` or the working directory
- [] `git reflog` still shows the discarded commit exists, proving nothing was truly lost

Expected Result

A clean, readable commit history on `messy-history`, with proof via `git reflog` that Git's safety net was there the whole time.

Extension Challenge

Research `git rebase -i` (interactive rebase) as a stretch goal — it lets you reorder, combine, and edit multiple past commits at once, not just the most recent one.

LEVEL 7 | Final Challenge

Prove you can run a real, multi-day workflow independently

Final Project: Multi-Branch Feature Release

CHALLENGE

CHALLENGE — minimal guidance from here. You've been given every tool needed; this project only names the destination.

Objective

Simulate a small, realistic release cycle for a fictional project of your choosing (a to-do app, a personal site, anything simple), using nearly every skill from this course.

Requirements

- A GitHub repository with a proper `.gitignore` from the start
- At least three feature branches, each rebased onto `main` before merging
- At least one intentionally resolved merge conflict
- At least one mistake deliberately made and fixed using `reset`, `revert`, or `amend`
- A tagged release (`v1.0.0`) once all features are merged
- A clean, readable history visible via `git log --oneline --graph --all`

Step-by-Step Guidance

Minimal — this is intentional. You know every command needed; the challenge is sequencing them correctly and recovering from your own mistakes without a script to follow.

- Plan your three features before starting — what are they, conceptually?
- Work through each one using the feature-branch pattern from Lesson 7.
- Deliberately introduce and resolve at least one conflict and one mistake somewhere along the way — don't wait for an accident.

Testing Checklist

- [] `.gitignore` exists and is committed before any junk files could be tracked
- [] All three features are merged into `main` with linear, rebased history
- [] A resolved conflict is visible somewhere in the log
- [] `v1.0.0` tag exists and is pushed
- [] You can narrate the entire history out loud, commit by commit, without checking notes

Expected Result

A small but completely real project history that looks like something a professional team could have produced — because at this scale, it essentially is.

Extension Challenge

Open a Pull Request on GitHub for one of your feature branches instead of merging locally, and use GitHub's web interface to complete the merge — this is how most real teams merge day to day.

Final Exam

This tests whether you can use Git, not just recite facts about it. Attempt every part before checking the answer key.

Part 1 — Concept Questions

- 1 What is the difference between the working directory, the staging area, and the repository?
- 2 What is a branch, technically?
- 3 What causes a merge conflict?
- 4 What's the difference between ``push``, ``pull``, and ``fetch``?
- 5 What's the difference between ``reset`` and ``revert``, and when should each be used?

Part 2 — Explain in Your Own Words

- 1 Explain rebase vs. merge to someone who has never used Git, using an analogy of your own choosing.
- 2 Explain why ``.gitignore`` matters, using a real example of a file that should never be committed.

Part 3 — Practical Problems

- 1 You have three uncommitted files, but only want to commit two of them together. Write the exact commands.
- 2 Write the commands to create a branch called ``fix/typo``, make one commit on it, and merge it into ``main``.
- 3 Write the commands to tag the current commit ``v2.0.0`` and push that tag.

Part 4 — Scenario Questions

- 1 You and a teammate both edited the same paragraph of the same file on different branches. Describe exactly what will happen when you merge, and what you'll need to do.
- 2 You rebased a branch, then remembered a teammate already pulled it. What's the risk, and what should you have done instead?

Part 5 — Troubleshooting

- 1 ``git push`` fails with "failed to push some refs." What's the likely cause and the fix?
- 2 You accidentally ran ``git reset --hard`` and lost a commit you needed. What command helps you find it again?
- 3 A ``.env`` file with real secrets was committed by mistake. What should you check/do (conceptually) beyond just adding it to ``.gitignore`` now?

Part 6 — Final Project

Your completed Final Project (Level 7) counts as Part 6 of this exam. If you haven't done it yet, complete it before considering the exam finished.

Final Exam — Answer Key

Part 1

1. The working directory is your actual files on disk as you edit them; the staging area is a preview holding area for what your next commit will contain; the repository is the permanent, saved commit history.
2. A branch is a lightweight, movable pointer to a specific commit — it doesn't copy any files.
3. Two branches changing the exact same lines of the exact same file — Git can't automatically decide which version to keep.
4. Push sends your local commits to a remote; pull downloads AND merges a remote's new commits into your current branch; fetch only downloads, without merging.
5. ``reset`` moves your branch pointer and rewrites local history — safe only on commits nobody else has; ``revert`` creates a new commit that undoes an old one — safe on shared/pushed history because nothing is rewritten.

Part 2

1. (Open-ended — check that the analogy correctly conveys: merge keeps both original paths visible and joins them, while rebase replays one path's commits fresh on top of the other, producing one straight line.)
2. (Open-ended — a strong answer names a specific example like a ``.env`` file with passwords, or a ``node_modules/`` folder, and explains that committing it either leaks secrets or bloats the repository with regenerable files.)

Part 3

1. ``git add file1 file2`` then ``git commit -m "message"`` — leaving the third file unstaged.
2. ``git switch -c fix/typo``, edit + ``git add .`` + ``git commit -m "...``, then ``git switch main`` and ``git merge fix/typo``.
3. ``git tag v2.0.0`` then ``git push origin v2.0.0``.

Part 4

1. Git will pause the merge and mark the file as conflicted, inserting ``<<<<<<<`/`=====`/`>>>>>>>`` markers showing both versions. You'll need to manually edit the file to choose or combine the content, delete the markers, then ``git add`` and commit to complete the merge.
2. The risk is that the teammate's local history now disagrees with the rewritten branch, producing duplicate/conflicting commits when they next sync. You should have avoided rebasing once the branch was shared — a plain merge would have been the safer choice at that point.

Part 5

1. Someone else pushed commits you don't have locally yet. Fix: ``git pull`` (resolving any conflicts), then ``git push`` again.
2. ``git reflog`` — it shows recent HEAD positions, including the commit from before the reset, which you can recover with ``git reset --hard ``.
3. Beyond adding it to ``.gitignore`` going forward, the secret is still present in the project's committed history and must be treated as compromised — it typically needs to be rotated/changed at the source, and the file should be removed from history using a dedicated history-rewriting tool if the repo is shared or public.

Final "Can I Actually Do This?" Checklist

Statement	✓
I can explain what Git is and why version control exists, without notes.	☐
I can run the init → add → commit → status → log loop from memory.	☐
I can create, switch between, and merge branches confidently.	☐
I can read and resolve a merge conflict by hand.	☐
I can clone, push, pull, and fetch from a remote.	☐
I understand <code>.gitignore</code> , <code>diff</code> , and <code>stash</code> and use them naturally.	☐
I can explain rebase vs. merge and the golden rule of rebasing.	☐
I know when to use <code>reset</code> vs. <code>revert</code> , and can use <code>reflog</code> as a safety net.	☐
I can create and push a tag.	☐
I completed the Final Project independently.	☐
I know which specific topics I still need to practice (list them below).	☐

Topics I still need to practice: _____

Glossary

Repository (repo)

Definition: A project folder Git is tracking the history of. **Example:** ``git init`` creates one. **Related:** Working directory, commit

Working directory

Definition: The actual files on disk as you edit them. **Example:** Editing `code.py` in your editor. **Related:** Staging area

Staging area

Definition: A preview holding area for your next commit. **Example:** ``git add file.py`` **Related:** `git add`, `git commit`

Commit

Definition: A saved snapshot of the project with a message. **Example:** ``git commit -m "...`"` **Related:** `git log`, HEAD

Branch

Definition: A movable pointer to a commit, an independent line of work. **Example:** ``git switch -c feature-x`` **Related:** HEAD, merge, rebase

HEAD

Definition: A pointer to your currently checked-out commit/branch. **Example:** Moves when you switch or commit. **Related:** Branch, checkout

Merge

Definition: Combining another branch's commits into your current branch. **Example:** ``git merge feature-x`` **Related:** Merge conflict, fast-forward

Merge conflict

Definition: When Git can't auto-combine two changes to the same lines. **Example:** Shown with `<<<<<<<` markers. **Related:** Merge, resolve

Rebase

Definition: Replaying a branch's commits on top of another branch's tip. **Example:** ``git rebase main`` **Related:** Golden rule, linear history

Remote

Definition: A hosted copy of your repo (e.g. on GitHub) your local repo syncs with. **Example:** Called ``origin`` by default. **Related:** Push, pull, fetch, clone

Clone

Definition: Downloading a full copy of an existing remote repository. **Example:** ``git clone`` **Related:** Remote

Push / Pull / Fetch

Definition: Send / (download+merge) / download-only, relative to a remote. **Example:** ``git push``, ``git pull``, ``git fetch`` **Related:** Remote

Reset

Definition: Moves the branch pointer, optionally discarding commits/changes. **Example:** ``git reset --hard HEAD~1`` **Related:** Revert, reflog

Revert

Definition: Creates a new commit that undoes an earlier one. **Example:** ``git revert `` **Related:** Reset (safe on shared history)

Reflog

Definition: A log of recent HEAD positions, including "lost" commits. **Example:** ``git reflog`` **Related:** Reset, safety net

Tag

Definition: A permanent, non-moving pointer to one specific commit. **Example:** ``git tag v1.0.0`` **Related:** Release, SemVer

.gitignore

Definition: A file listing patterns Git should never track. **Example:** ``__pycache__/, .env`` **Related:** Staging area

Stash

Definition: Temporary storage for uncommitted changes. **Example:** ``git stash`, `git stash pop`` **Related:** Working directory

Cheat Sheet

A one-stop review of the entire course. Skim this whenever you need a fast refresher.

Setup

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Core Loop

```
git init
git status
git add <file>          # or: git add .
git commit -m "message"
git log --oneline
```

Branching & Merging

```
git branch              # list branches
git switch -c <name>    # create + switch
git switch <name>       # switch
git merge <name>        # merge into current branch
git branch -d <name>    # delete (safe, only if merged)
```

Remotes

```
git clone <url>
git remote -v
git remote add origin <url>
git push -u origin main
git push
git pull
git fetch
```

Everyday Tools

```
git diff                # unstaged changes
git diff --staged       # staged changes
git stash / git stash pop
echo "pattern" >> .gitignore
```

Rebase

```
git rebase main          # replay current branch's commits onto main
git rebase --continue    # after resolving a conflict
git rebase --abort       # cancel
```

COMMON MISTAKE

Golden rule: never rebase commits already pushed/shared with others.

Undoing

```
git restore <file>           # discard uncommitted edit
git restore --staged <file>   # unstage
git reset --soft HEAD~1      # undo commit, keep changes staged
git reset --hard HEAD~1      # undo commit, discard changes (local only!)
git revert <hash>             # safe undo via new commit (shared history)
git commit --amend -m "... " # fix the most recent commit's message
git reflog                   # recover "lost" commits
```

Tags

```
git tag v1.0.0
git push origin v1.0.0
git push origin --tags
```

Common Mistakes At a Glance

Mistake	Fix
Committed with nothing staged	`git add` first, then `git commit`
Push rejected	`git pull`, resolve conflicts, `git push` again
Rebased shared history	Don't — use `merge` once a branch is shared
Lost a commit after reset --hard	`git reflog`, then `git reset --hard <hash>`
Secret committed by mistake	.gitignore it going forward AND rotate the secret

What To Learn Next

If you enjoyed this subject

- Interactive rebase (`git rebase -i`) — reorder, squash, and edit multiple commits at once.
- `git bisect` — binary-search your history to find exactly which commit introduced a bug.

If you want to become job-ready

- Pull Request workflows on GitHub/GitLab — code review, comments, required checks.
- Continuous Integration (CI) basics — how pushes/PRs can trigger automated tests.
- Writing good commit messages and PR descriptions as a team communication skill.

If you want to specialize

- Git internals — how commits, trees, and blobs are actually stored as objects.
- Large-scale monorepo tooling (e.g. sparse checkout, submodules, subtrees).
- Git hooks — running scripts automatically on commit/push for linting or tests.

If you want to go deeper

- `git cherry-pick` in depth — selectively applying individual commits across branches.
- Signed commits and GPG verification for security-conscious projects.
- Given your interest in backend and full-stack AI engineering work, pairing this Git foundation with a FastAPI project — branching per feature, tagging releases, and opening real Pull Requests — will cement everything in this course fast.

Course complete. Go build something and break it on purpose — that's still the fastest way to learn Git.